

13주 차 보고서

(2026년 3 월 22 일 ~ 2026년 3 월 28 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

전에 강의를 공부하면서 임의로 만들었던 프로토콜을 우리의 게임에 맞게 다시 제대로 만들고, 하나씩 추가할 예정이다.

```

syntax = "proto3";
package Protocol;

enum ObjectType
{
    OBJECT_TYPE_NONE = 0;
    OBJECT_TYPE_CREATURE = 1;      // 유저, 좀비
    OBJECT_TYPE_PROJECTILE = 2;    // 총알, 날라가는 투사체
    OBJECT_TYPE_ITEM = 3;         // 바닥에 있는 아이템
}

enum CreatureType
{
    CREATURE_TYPE_NONE = 0;
    CREATURE_TYPE_PLAYER = 1;
    CREATURE_TYPE_ZOMBIE = 2;
}

enum PlayerType
{
    PLAYER_TYPE_NONE = 0;
    PLAYER_TYPE_SHOOTER = 1;
}

enum ZombieType
{
    ZOMBIE_TYPE_NONE = 0;
    ZOMBIE_TYPE_MELEE = 1;
    ZOMBIE_TYPE_RANGED = 2;
    ZOMBIE_TYPE_TANKER = 3;
}

enum MoveState
{
    MOVE_STATE_NONE = 0;
    MOVE_STATE_IDLE = 1;
    MOVE_STATE_RUN = 2;
    MOVE_STATE_JUMP = 3;
    MOVE_STATE_FALL = 4;      // 점프 후 떨어질때
    MOVE_STATE_DEAD = 6;
}

enum WeaponType
{
    WEAPON_TYPE_NONE = 0;
    WEAPON_TYPE_RIFLE = 1;
}

```

생물체는 플레이어와 좀비밖에 없어서 변동이 없을 예정이고, 아이템은 여러가지가 있는데 아직은 총밖에 없어서 추후에 추가할 예정이다.

```

message C_EQUIP_WEAPON
{
    uint64 itemObjectId = 1;
}

message S_EQUIP_WEAPON
{
    uint64 playerId = 1;      // 누가 주었는지 (이 사람 손에 총을 붙여야 함)
    uint64 itemObjectId = 2;  // 맵에 떨어져 있던 어떤 아이템을 주었는지 (주었으니 맵에서 지워야 함)
    int32 weaponType = 3;    // 그 아이템이 무슨 총인지 (어떤 모델링을 띄울지 결정)
}

```

총을 주웠을때에 대한 프로토콜도 만들어주었다.

```

message PosInfo
{
    uint64 object_id = 1;
    float x = 2;
    float y = 3;
    float z = 4;
    float yaw = 5;
    MoveState state = 6;
}

message ObjectInfo
{
    uint64 object_id = 1;    // 고유 id
    ObjectType object_type = 2; // 유저인지 좀비인지 아이템인지
    PosInfo pos_info = 3;    // 현재 위치 및 상태
    int32 weapon_type = 4;
}

```

아직은 이동과 총만 있다.



이번주에는 총을 주웠을 때 상대방 화면에서 총을 주운 모습이 제대로 보이도록 했다. 처음에는 총이 이상한 위치에 착용이 되어서 혼자서 각도를 조정해보았는데



여전히 이상한 위치에 있어서 클라 친구에게 어떻게 위치를 조절했는지 묻고 바로 해결했다.

```
void AFPSBaseCharacter::EquipWeaponFromField(AWeaponBase* Weapon)
{
    if (Weapon == nullptr) return;

    // 1. 바닥에 고정되어 있던 물리나 충돌을 끄기
    Weapon->SetActorEnableCollision(false);

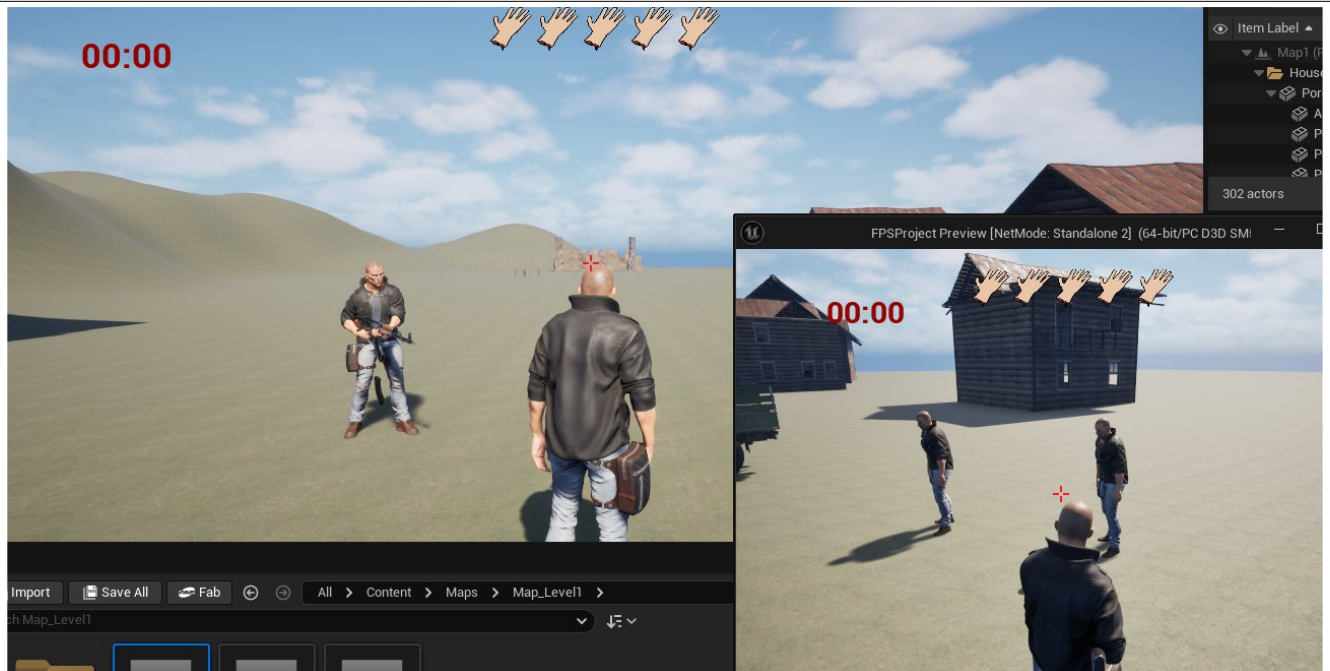
    // 2. [소켓에 부착 (SnapToTarget을 써야 소켓 위치로 순간이동합니다.)
    const FName SocketName = TEXT("Gun_socket");
    Weapon->AttachToComponent(Parent::GetMesh(), FAttachmentTransformRules::SnapToTargetNotIncludingScale, SocketName);
    // 3. 위치 조정 (Location)
    Weapon->SetActorRelativeLocation(FVector(InX:-35.209697f, InY:2.353551f, InZ:0.508678f));

    // 회전 조정 (Rotation - Pitch, Yaw, Roll 순서)
    Weapon->SetActorRelativeRotation(FRotator(InPitch:1.090108f, InYaw:-88.966904f, InRoll:-4.015320f));

    // 스케일 조정 (Scale) - 총이 0.15배로 작아져야 하니까!
    Weapon->SetActorRelativeScale3D(FVector(InX:0.15f, InY:0.15f, InZ:0.15f));

    // 4. 캐릭터 변수 업데이트 및 애니메이션Instance 변경
    SetCurrentWeapon(Weapon);

    UE_LOG(LogTemp, Log, TEXT("[Network] %s가 바닥에 있던 무기(%s)를 장착했습니다."), *GetName(), *Weapon->GetName());
}
```

역시 모르는게 있으면 바로바로 물어봐야한다.

```
bool Handle_C_EQUIP_WEAPON(PacketSessionRef& session, Protocol::C_EQUIP_WEAPON& pkt)
{
    GameSessionRef gameSession = static_pointer_cast<GameSession>(session);
    PlayerRef player = gameSession->player.load();
    if (player == nullptr)
        return false;

    // 플레이어가 속한 방(Room)을 찾을
    RoomRef room = player->room.load().lock();
    if (room == nullptr)
        return false;

    // 방에 넣길 때 임시 객체로 복사
    room->DoAsync(&Room::HandleEquipWeapon, player, Protocol::C_EQUIP_WEAPON(pkt));

    return true;
}
```

```
bool Handle_S_EQUIP_WEAPON(PacketSessionRef& session, Protocol::S_EQUIP_WEAPON& pkt)
{
    if (auto* GameInstance = Cast<UFPSProjectGameInstance>(Src: GWorld->GetGameInstance()))
    {
        //일단 서버를 돌아서 내 클라까지 패킷이 잘 도착했는지 로그로 확인!
        //UE_LOG(LogTemp, Warning, TEXT("===== [네트워크] S_EQUIP_WEAPON 수신 ====="));
        //UE_LOG(LogTemp, Warning, TEXT("누가 주웠는가(PlayerID) : %llu"), pkt.playerid());
        //UE_LOG(LogTemp, Warning, TEXT("무슨 아이템(ItemID) : %llu"), pkt.itemobjectid());
        //UE_LOG(LogTemp, Warning, TEXT("무기 타입(WeaponType) : %d"), pkt.weaponType());

        // GameInstance에 함수를 만들어서 실제 모델링을 손에 붙이기
        GameInstance->HandleEquipWeapon(pkt);
    }

    return true;
}
```

패킷 송수신을 확인한 후에

```
void UFPSProjectGameInstance::HandleEquipWeapon(const Protocol::S_EQUIP_WEAPON& pkt)
{
    uint64 PlayerId = pkt.playerid();
    uint64 ItemId = pkt.itemobjectid();

    // 누가 주었는지 찾기
    AFPSBaseCharacter* TargetPlayer = Players.Contains(PlayerId) ? Players[PlayerId] : nullptr;

    // 바닥에 있는 총 찾기 (FieldItems 맵에 등록해둔 것)
    if (FieldItems.Contains(ItemId))
    {
        AWeaponBase* WeaponActor = Cast<AWeaponBase>(FieldItems[ItemId]);

        if (TargetPlayer && WeaponActor)
        {
            // 내 캐릭터라면 이미 로컬에서 처리가 되었겠지만,
            // 혹시 모를 동기화를 위해 남의 캐릭터일 때만 실행
            if (!TargetPlayer->IsLocallyControlled())
            {
                TargetPlayer->EquipWeaponFromField(WeaponActor);
            }

            // 이제 바닥에 없으니 관리 목록에서 제거!
            FieldItems.Remove(ItemId);
        }
    }
}
```

```
void Room::Broadcast(SendBufferRef sendBuffer, uint64 exceptId)
{
    for (auto& item : pair<...> : _objects)
    {
        PlayerRef player = dynamic_pointer_cast<Player>(item.second);
        if (player == nullptr)
            continue;
        if (player->objectInfo->object_id() == exceptId)
            continue;

        if (GameSessionRef session = player->session.lock())
            session->Send(sendBuffer);
    }
}
```

상대방 화면에서도 총이 정확한 위치와 각도로 부착되어서 렌더링 되도록 동기화 처리를 했다.

다음 주 할 일

플레이어가 차량 앞자리에 탑승했을 때와 트렁크에 탑승했을 때의 위치를 보내줄 예정이다.